Web Analyzer

System Overview

The Web Analyzer is a Go-based web service that analyzes HTML pages by extracting metadata, detecting HTML versions, counting headings, analyzing links (internal/external/inaccessible), and detecting login forms. The system implements caching, monitoring, and performance profiling capabilities.

Architecture

System Components

1. Web Analyzer Server (Port 8080)

- Main HTTP server handling analysis requests
- Implements RESTful API with versioning
- Middleware stack for security, logging, compression, CORS, rate limiting, and metrics

2. Cache Layer (In-Memory)

- Go-based in-memory cache with TTL support
- Reduces redundant external HTTP requests
- 1-hour cache expiration for analysis results

3. Metrics Server (Port 8081)

- Prometheus-compatible metrics endpoint
- Separate server for operational monitoring
- Collects request counts, latencies, and error rates

4. Pprof Server (Port 6061, Development Only)

- Performance profiling via Go's pprof package
- CPU and memory profiling capabilities
- Disabled in production environments

Key Design Decisions

1. Concurrent Analysis with Goroutines

Use 5 concurrent goroutines in AnalyzePage for parallel processing of HTML analysis tasks.

Rationale:

- HTML version detection, title extraction, heading counts, link analysis, and login form detection are independent operations
- Parallel execution reduces total analysis time
- sync.WaitGroup ensures all goroutines complete before returning results

Trade-offs:

- Increased memory usage per request
- Potential race conditions (mitigated by proper synchronization)

2. Worker Pool for Link Accessibility Checks

Implement a dynamic worker pool for checking link accessibility.

Rationale:

- Pages can contain hundreds of links
- Sequential checking would be prohibitively slow
- Worker pool limits concurrent HTTP requests to prevent resource exhaustion

3. Fallback HEAD to GET Requests

Attempt HEAD request first, fallback to GET if HEAD fails.

Rationale:

- HEAD requests are lighter (no body transfer)
- Some servers don't support HEAD or respond differently
- GET ensures comprehensive link validation

4. Separate Metrics Server

Decision: Run Prometheus metrics on a separate port (8081) from the main API (8080).

Rationale:

- Security isolation (metrics may contain sensitive operational data)
- Prevents metrics scraping from affecting API performance
- Allows different access control policies

5. Graceful Shutdown

Implement 5-second timeout for graceful shutdown.

Rationale:

- Allows in-flight requests to complete
- Prevents data loss or corrupted responses
- Balances cleanup time vs. deployment speed

Assumptions

Technical Assumptions

1. **Target websites are publicly accessible** - No authentication/proxy support implemented
2. **HTML content is UTF-8 encoded** - Character encoding detection not implemented
3. **Maximum 30-second timeout** for fetching pages and checking links
4. **Link analysis accuracy** - Assumes 3 redirects maximum before considering link problematic
5. **Login form detection heuristics** - Based on keyword matching (`login`, `sign in`, `password`, etc.)

Operational Assumptions

1. **Single-instance deployment** - In-memory cache is not distributed
2. **Moderate traffic** - Rate limiting configured
3. **Target sites allow scraping** - No robots.txt checking or rate limiting to target sites
4. **Network reliability** - No retry logic for failed HTTP requests

Challenges & Solutions

Preventing Resource Exhaustion

Analyzing pages with thousands of links could overwhelm the system.

Solution:

- Dynamic worker pool sizing based on link count
- Context timeout (30 seconds) for link analysis operations
- Early termination via context cancellation

Login Form False Positives/Negatives

Keyword-based detection may miss non-standard forms or flag search forms.

Solution:

- Multi-signal approach: checks for password inputs AND login keywords
- Searches within form scope only (not entire page)
- Extensible keyword list

Improvements

1. Distributed Caching

In memory cache lost on restart. not shared across instances

Solution: Integrate Redis
Benefit: Persistent cache, horizontal scalability

2. Configurable Timeouts

Currently hardcoded.Move to environment variables or config files.

3. JavaScript Rendering

Cannot analyze Single Page Applications or dynamically loaded content

Solution: Integrate headless browser
Trade-off: Increased resource usage and latency

4. Request Queueing

For high-traffic scenarios, implement message queue (RabbitMQ, Kafka)
Decouple request receipt from processing

5. Machine Learning for Login Detection

- Train model on labeled form data.
- More accurate than keyword matching.
- Reduces maintenance of keyword lists.

6.Historical Analysis Tracking

- Store analysis results in database (PostgreSQL, MongoDB)
- Enable trend analysis
- Support bulk analysis and reporting

Security Considerations

- Rate limiting - Prevents abuse
- URL validation - Blocks malformed inputs
- Secure headers middleware - Sets security headers (likely CSP, X-Frame-Options)
- CORS middleware - Controls cross-origin access
- Panic recovery - Prevents crashes from exposing internal state

API Documentation

Endpoint: Analyze Page

Request:

http

GET /webanalyzer/api/v1/analyze?url=https://example.com

Response (Success):

json


```
{
  "status": "OK",
  "status_code": 200,
  "data": {
    "html_version": "HTML5",
    "page_title": "Example Domain",
    "heading_counts": {
      "h1": 1,
      "h2": 3,
      "h3": 5,
      "h4": 0,
      "h5": 0,
      "h6": 0
    },
    "internal_link_count": 12,
    "external_link_count": 5,
    "inaccessible_link_count": 2,
    "has_login_form": false
  }
}
```

Response (Error):

Json


```
{
  "status": "Bad Gateway",
  "status_code": 502,
  "message": "failed to analyze page: unexpected status code: 403"
}
```


Status Codes:

- `200 OK` - Analysis successful
- `400 Bad Request` - Invalid or missing URL
- `502 Bad Gateway` - Target site unreachable
- `504 Gateway Timeout` - Analysis timed out
- `500 Internal Server Error` - Server error